

Organização e Arquitetura de Computadores

Prof. Dr. Marcus Vinicius Lamar

Laboratório 1

Grupo 13 – 2026/1

Danilo Lins Castro – 251019735 – 251019735@aluno.unb.br

Eduardo Sankievicz Lima – 251035102 – 251035102@aluno.unb.br

Fernando de Almeida Mendes Dias – 251040701 – 251040701@aluno.unb.br

Helton Rodrigues da Silva Júnior – 251021789 – 251021789@aluno.unb.br

João Pedro Barros Silva Azevedo – 251004311 – 251004311@aluno.unb.br

João Vitor da Silva Lima – 251023101 – 251023101@aluno.unb.br

1 Simulador/Montador RARS

1.1 Item 1.2

a) Considera-se a execução do algoritmo do programa `sort.s` em um processador RISC-V com frequência de clock de 50 MHz, o qual necessita de 1 ciclo de clock para a execução de cada instrução ($CPI = 1$).

Adotam-se como entrada dois vetores de tamanho n :

$$V_0[n] = \{1, 2, 3, 4, \dots, n\}$$

o qual possui os elementos já ordenados, e

$$V_i[n] = \{n, n-1, n-2, \dots, 2, 1\}$$

o qual possui os elementos ordenados na ordem inversa.

Tendo isso em vista, para determinar a equação dos tempos de execução em função de n para ambos os vetores, utiliza-se a equação fundamental de desempenho:

$$t_{exec} \left[\frac{\text{segundos}}{\text{programa}} \right] = I \left[\frac{\text{Instruções}}{\text{programa}} \right] \times CPI \left[\frac{\text{Ciclos_clock}}{\text{Instrução}} \right] \times T \left[\frac{\text{segundos}}{\text{Ciclos_clock}} \right]$$

Substituindo-se os valores do número de ciclos por instrução e do tempo de ciclo do processador RISC-V, tem-se:

$$t_{exec} = I \cdot 1 \cdot \frac{1}{50 \times 10^6}$$

Logo,

$$t_{exec} = \frac{I}{50 \times 10^6}$$

Dessa forma, entende-se que o tempo de execução do programa `sort.s` é proporcional à quantidade de instruções executadas, a qual pode ser determinada a partir do vetor de entrada.

Além disso, nota-se que a quantidade de instruções executadas depende tanto do tamanho quanto da ordenação do vetor. Primeiramente, obtém-se a função que determina o número de instruções para a entrada $V_0[n]$.

```

SORT:  addi sp,sp,-20
        sw ra,16(sp)
        sw s3,12(sp)
        sw s2,8(sp)
        sw s1,4(sp)
        sw s0,0(sp)
        mv s2,a0
        mv s3,a1
        mv s0,zero
for1:   bge s0,s3,exit1
        addi s1,s0,-1
for2:   blt s1,zero,exit2
        slli t1,s1,2
        add t2,s2,t1
        lw t3,0(t2)
        lw t4,4(t2)
        bgt t4,t3,exit2
        mv a0,s2
        mv a1,s1
        jal SWAP
        addi s1,s1,-1
        j for2
exit2:  addi s0,s0,1
        j for1
exit1:  lw s0,0(sp)
        lw s1,4(sp)
        lw s2,8(sp)
        lw s3,12(sp)
        lw ra,16(sp)
        addi sp,sp,20
        ret

```

Figura 1: Código da função `sort.s`.

Ao analisar o código, percebe-se que, para um vetor de n elementos já ordenados, a função de contagem de instruções segue a seguinte forma:

$$I_0(n) = n \cdot (\text{For1} + \text{For2} + \text{Exit2}) + K - 5$$

Sob a perspectiva de um vetor de entrada já ordenado, o termo K é determinado pelo trecho compreendido entre as instruções `addi sp, sp, -20` e `mv s0, zero`, bem como pela instrução `exit1`, totalizando 17 instruções executadas independentemente do valor de n .

Além disso, para cada iteração do laço externo, o programa executa integralmente os blocos `for1` e `exit2`. Já o bloco `for2` é executado apenas até a instrução `bgt t4, t3, exit2`, visto que, em um vetor previamente ordenado, o valor armazenado em `t4` será sempre maior que o valor armazenado em `t3`. Dessa forma, cada iteração contribui com um total de 10 instruções.

Por fim, o termo -5 justifica-se pelo fato de que, para $i = 0$, o bloco `for2` executa apenas sua primeira instrução, reduzindo a contagem total esperada.

Assim, substituindo-se os valores obtidos na equação anterior, tem-se:

$$I_0(n) = n \cdot (10) + 17 - 5$$

Logo,

$$I_0(n) = 10n + 12$$

Substituindo a expressão obtida na equação do tempo de execução:

$$t_{exec}(n) = \frac{10n + 12}{50 \times 10^6}$$

```

SWAP:      slli t1,a1,2
              add t1,a0,t1
              lw  t0,0(t1)
              lw  t2,4(t1)
              sw  t2,0(t1)
              sw  t0,4(t1)
              ret

```

Figura 2: Código do bloco swap swap.

Agora, analisa-se o algoritmo de ordenação para o pior caso, portanto, adota-se como entrada um vetor inversamente ordenado com n elementos, obtém-se a seguinte função por meio da análise algorítmica:

$$I_i(n) = K + n \cdot (\text{For1} + \text{Exit2}) + n \cdot (\text{TesteSaídaFor2}) + \frac{n(n-1)}{2} \cdot (\text{Troca})$$

Nota-se que, para o vetor V_i , a quantidade de instruções constantes permanece a mesma, isto é, $K = 17$. Além disso, sabe-se que os blocos **For1** e **Exit2** totalizam 4 instruções executadas a cada iteração do laço externo.

Ademais, o bloco **TesteSaídaFor2** é definido por apenas uma instrução, **blt s1, zero**, responsável pela verificação da condição de saída do segundo laço.

Por fim, o bloco **Troca** é caracterizado pelas instruções presentes em **For2** responsáveis pela verificação de que $V[i+1]$ é maior que $V[i]$, bem como pela execução da função **SWAP**, totalizando 18 instruções.

Dessa forma, substituindo-se os valores obtidos na equação anterior, tem-se:

$$I_i(n) = 17 + 4n + n + \frac{n(n-1)}{2} \cdot 18$$

Desenvolvendo a expressão:

$$I_i(n) = 17 + 5n + 9n(n-1)$$

$$I_i(n) = 17 + 5n + 9n^2 - 9n$$

Logo,

$$I_i(n) = 9n^2 - 4n + 17$$

Substituindo a expressão obtida na equação do tempo de execução:

$$t_{exec}(n) = \frac{9n^2 - 4n + 17}{50 \times 10^6}$$

b)

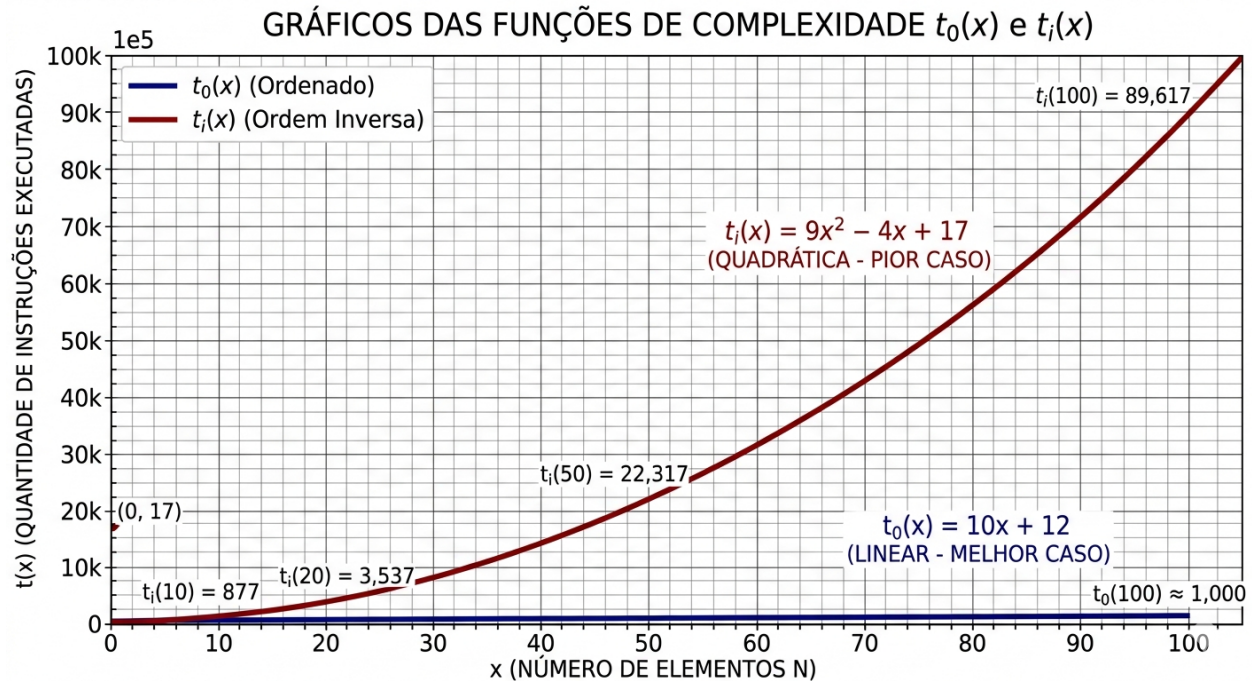


Figura 3: Gráfico comparativo das funções $I_i(x)$ e $I_0(x)$.

Sob essa perspectiva, ao observar as curvas das funções $I_i(n)$ e $I_0(n)$, conclui-se que, no melhor caso, o algoritmo de ordenação `sort.s` possui complexidade $O(n)$, enquanto, no pior caso, apresenta complexidade $O(n^2)$.

2 Compilador cruzado GCC

2.1 Compilação com as diretivas -S -O0

Realizando a compilação do arquivo `sortc.c` disponibilizado nos arquivos do Lab 1 com as diretivas `-S -O0`, obtemos um código Assembly padrão, sem nenhuma otimização.

```

1 #Codigo O0 gerado
2 #Alteracao dos 'lla' por 'la':
3 #Correção dos return: GCC termina a 'main' com um 'jr ra'.
4 #Foi trocado pela Syscall de encerramento ('li a7, 10' e 'ecall'), senão o
   Rars daria *runtime exception*.
5 .data    #Foi adicionado o .data
6 v:
7     .word    9
8     .word    2
9     .word    5
10    .word    1
11    .word    8

```

```

12      .word    2
13      .word    4
14      .word    3
15      .word    6
16      .word    7
17      .word   10
18      .word    2
19      .word   32
20      .word   54
21      .word    2
22      .word   12
23      .word    6
24      .word    3
25      .word    1
26      .word   78
27      .word   54
28      .word   23
29      .word    1
30      .word   54
31      .word    2
32      .word   65
33      .word    3
34      .word    6
35      .word   55
36      .word   31
37
38      .text #tambem foi necessário adicionar .text
39      .globl main # de onde o rars irá começar a rodar
40      j main #salto para a main
41
42      #remocao de toda a estrutura antiga da label show, juntamente com o .LC0,.L3 e
43      .L2
44      swap:
45          addi    sp,sp,-48
46          sw      ra,44(sp)
47          sw      s0,40(sp)
48          addi    s0,sp,48
49          sw      a0,-36(s0)
50          sw      a1,-40(s0)
51          lw      a5,-40(s0)
52          slli    a5,a5,2
53          lw      a4,-36(s0)
54          add     a5,a4,a5
55          lw      a5,0(a5)
56          sw      a5,-20(s0)
57          lw      a5,-40(s0)
58          addi    a5,a5,1
59          slli    a5,a5,2
60          lw      a4,-36(s0)
61          add     a4,a4,a5
62          lw      a5,-40(s0)
63          slli    a5,a5,2
64          lw      a3,-36(s0)
65          add     a5,a3,a5

```

```

65      lw      a4,0(a4)
66      sw      a4,0(a5)
67      lw      a5,-40(s0)
68      addi    a5,a5,1
69      slli    a5,a5,2
70      lw      a4,-36(s0)
71      add     a5,a4,a5
72      lw      a4,-20(s0)
73      sw      a4,0(a5)
74      nop
75      lw      ra,44(sp)
76      lw      s0,40(sp)
77      addi    sp,sp,48
78      jr      ra
79 sort:
80      addi    sp,sp,-48
81      sw      ra,44(sp)
82      sw      s0,40(sp)
83      addi    s0,sp,48
84      sw      a0,-36(s0)
85      sw      a1,-40(s0)
86      sw      zero,-20(s0)
87      j       .L6
88 .L10:
89      lw      a5,-20(s0)
90      addi    a5,a5,-1
91      sw      a5,-24(s0)
92      j       .L7
93 .L9:
94      lw      a1,-24(s0)
95      lw      a0,-36(s0)
96      call    swap
97      lw      a5,-24(s0)
98      addi    a5,a5,-1
99      sw      a5,-24(s0)
100 .L7:
101      lw      a5,-24(s0)
102      blt     a5,zero,.L8
103      lw      a5,-24(s0)
104      slli    a5,a5,2
105      lw      a4,-36(s0)
106      add     a5,a4,a5
107      lw      a4,0(a4)
108      lw      a5,-24(s0)
109      addi    a5,a5,1
110      slli    a5,a5,2
111      lw      a3,-36(s0)
112      add     a5,a3,a5
113      lw      a5,0(a5)
114      bgt     a4,a5,.L9
115 .L8:
116      lw      a5,-20(s0)
117      addi    a5,a5,1
118      sw      a5,-20(s0)

```

```

119 .L6:
120     lw      a4,-20(s0)
121     lw      a5,-40(s0)
122     blt     a4,a5,.L10
123     nop
124     nop
125     lw      ra,44(sp)
126     lw      s0,40(sp)
127     addi    sp,sp,48
128     jr      ra
129
130 main:
131     addi    sp,sp,-16
132     sw      ra,12(sp)
133     sw      s0,8(sp)
134     addi    s0,sp,16
135     li      a1,30
136     la      a0,v
137     call    SHOW
138     li      a1,30
139     la      a0,v
140     call    sort
141     li      a1,30
142     la      a0,v
143     call    SHOW
144     li      a5,0
145     mv      a0,a5
146     lw      ra,12(sp)
147     lw      s0,8(sp)
148     addi    sp,sp,16
149     call    SHOW
150     li      a7,10 # Prepara o encerramento do programa
151     ecall   # Executa a Syscall (CORRIGIDO)
152
153     #adicionado a funcao show do arquivo sort.s
154 SHOW:  mv    t0,a0
155         mv    t1,a1
156         mv    t2,zero

```

Listing 1: Código gerado usando as diretivas -S -O0

Com isso, vale ressaltar que foi utilizada a função `SHOW` do código `sort.s` disponibilizado nos arquivos do laboratório, com o intuito de não precisar implementar a função `printf` contida no arquivo em C.

2.2 Tabela comparativa

Após compilar o arquivo `sortc_mod.c` com as 3 diretivas de compilação indicadas (-O0, -O3, -Os) e transferir os códigos para o RARS, foi possível fazer uma análise breve do número de instruções e do tamanho em bytes de cada código para ressaltar suas diferenças e ainda fazer uma comparação com o código `sort.s`.

Versão	Número de Instruções	Tamanho em Bytes
-O0	20	536 bytes
-O3	8	432 bytes
-Os	8	328 bytes
sort.s	6220	276 bytes

Tabela 1: Comparação entre diferentes níveis de otimização

A partir dos dados acima, a diferença do mesmo código mas com diretivas de compilação diferentes fica clara, enquanto -O0 é o maior código sem nenhuma otimização, O3 e Os apresentam menos instruções e também menos bytes de tamanho.

2.3 Comparação de tempos de execução de funções

Utilizando o programa `teste12.c` disponibilizado nos arquivos do Lab 1, foram comparados, pelo software RARS, os tempos de execução de cada função contida nele, sendo elas: `f1`, `f2`, `f3`, `f4`, `f5` e `f6`, usando as diretivas -O0 e -O1.

Abaixo, seguem os códigos de `teste12.s` com as otimizações -O0 e -O1, juntamente com os valores de seus registradores correspondentes.

```

1  .text
2  .globl main
3  jal main
4
5  F1:
6      addi    sp, sp, -32
7      sw      ra, 28(sp)
8      sw      s0, 24(sp)
9      addi    s0, sp, 32
10     sw      a0, -20(s0)
11     lw      a5, -20(s0)
12     slli    a5, a5, 2
13     mv      a0, a5
14     lw      ra, 28(sp)
15     lw      s0, 24(sp)
16     addi    sp, sp, 32
17     jr      ra
18
19  F2:
20     addi    sp, sp, -32
21     sw      ra, 28(sp)
22     sw      s0, 24(sp)
23     addi    s0, sp, 32
24     sw      a0, -20(s0)
25     lw      a5, -20(s0)
26     slli    a5, a5, 2
27     mv      a0, a5
28     lw      ra, 28(sp)
29     lw      s0, 24(sp)
30     addi    sp, sp, 32
31     jr      ra
32
33  F3:
34     addi    sp, sp, -32
35     sw      ra, 28(sp)
36     sw      s0, 24(sp)
37     addi    s0, sp, 32

```

```

36      sw      a0,-20(s0)
37      lw      a5,-20(s0)
38      slli    a5,a5,2
39      mv      a0,a5
40      lw      ra,28(sp)
41      lw      s0,24(sp)
42      addi    sp,sp,32
43      jr      ra
44 F4:
45      addi    sp,sp,-32
46      sw      ra,28(sp)
47      sw      s0,24(sp)
48      addi    s0,sp,32
49      sw      a0,-20(s0)
50      lw      a5,-20(s0)
51      slli    a5,a5,2
52      mv      a0,a5
53      lw      ra,28(sp)
54      lw      s0,24(sp)
55      addi    sp,sp,32
56      jr      ra
57 F5:
58      addi    sp,sp,-32
59      sw      ra,28(sp)
60      sw      s0,24(sp)
61      addi    s0,sp,32
62      sw      a0,-20(s0)
63      lw      a5,-20(s0)
64      slli    a5,a5,2
65      mv      a0,a5
66      lw      ra,28(sp)
67      lw      s0,24(sp)
68      addi    sp,sp,32
69      jr      ra
70 F6:
71      addi    sp,sp,-32
72      sw      ra,28(sp)
73      sw      s0,24(sp)
74      addi    s0,sp,32
75      sw      a0,-20(s0)
76      lw      a5,-20(s0)
77      fcvt.d.w    fa4,a5
78      la      a5,.LC0
79      fld     fa5,0(a5)
80      fmul.d   fa5,fa4,fa5
81      fcvt.w.d a5,fa5,rtz
82      mv      a0,a5
83      lw      ra,28(sp)
84      lw      s0,24(sp)
85      addi    sp,sp,32
86      jr      ra
87
88 main:
89      csrr    s1,3074 # le o num instr atual

```

```

90  csrr s0,3073 # le o time atual
91  jal F1
92  csrr t0,3073 # le o time atual
93  csrr t1,3074 # le o num instr atual
94  sub s0,t0,s0 # calcula o tempo de execução / s0 = time
95  sub s1,t1,s1 # calcula o número de instruções executadas
96
97  csrr s3,3074
98  csrr s2,3073
99  jal F2
100 csrr t0,3073
101 csrr t1,3074
102 sub s2,t0,s2 # s2 = time
103 sub s3,t1,s3
104
105 csrr s5,3074
106 csrr s4,3073
107 jal F3
108 csrr t0,3073
109 csrr t1,3074
110 sub s4,t0,s4 # s4 = time
111 sub s5,t1,s5
112
113 csrr s7,3074
114 csrr s6,3073
115 jal F4
116 csrr t0,3073
117 csrr t1,3074
118 sub s6,t0,s6 # s6 = time
119 sub s7,t1,s7
120
121 csrr s9,3074
122 csrr s8,3073
123 jal F5
124 csrr t0,3073
125 csrr t1,3074
126 sub s8,t0,s8 # s8 = time
127 sub s9,t1,s9
128
129 csrr s11,3074
130 csrr s10,3073
131 jal F6
132 csrr t0,3073
133 csrr t1,3074
134 sub s10,t0,s10 # s10 = time
135 sub s11,t1,s11
136
137 li a7, 10
138 ecall
139
140 .data
141 .LC0:
142     .word    0
143     .word    1074790400

```

Listing 2: teste12.s usando a otimização -O0

Dada essa otimização, foi calculado o tempo de execução de cada função e armazenado nos registradores s0, s2, s4, s6, s8 e s10, na respectiva ordem das funções. Segue abaixo a imagem com os valores de cada registrador.

Registers	Floating Point	Control and Status
Name	Number	Value
zero	0	0
ra	1	4194768
sp	2	2147479548
gp	3	268468224
tp	4	0
t0	5	1179327415
t1	6	117
t2	7	0
s0	8	2491
s1	9	16
a0	10	0
a1	11	0
a2	12	0
a3	13	0
a4	14	0
a5	15	0
a6	16	0
a7	17	10
s2	18	2694
s3	19	16
s4	20	3861
s5	21	16
s6	22	2539
s7	23	16
s8	24	2593
s9	25	16
s10	26	3680
s11	27	21
t3	28	0
t4	29	0
t5	30	0
t6	31	0
pc		4194792

Figura 4: Registradores da otimização -O0

```

1 .text
2 .globl main
3 jal main
4
5 F1:
6     slli    a0, a0, 2
7     ret
8 F2:
9     slli    a0, a0, 2
10    ret
11 F3:
12    slli    a0, a0, 2
13    ret
14 F4:

```

```

15      slli    a0,a0,2
16      ret
17 F5:
18      slli    a0,a0,2
19      ret
20 F6:
21      fcvt.d.w    fa5,a0
22      fld      fa4,LC0,a5
23      fmul.d    fa5,fa5,fa4
24      fcvt.w.d  a0,fa5,rtz
25      ret
26
27 main:
28      csrr s1,3074 # le o num instr atual
29      csrr s0,3073 # le o time atual
30      jal F1
31      csrr t0,3073 # le o time atual
32      csrr t1,3074 # le o num instr atual
33      sub s0,t0,s0 # calcula o tempo de execução / s0 = time
34      sub s1,t1,s1 # calcula o número de instruções executadas
35
36      csrr s3,3074
37      csrr s2,3073
38      jal F2
39      csrr t0,3073
40      csrr t1,3074
41      sub s2,t0,s2 # s2 = time
42      sub s3,t1,s3
43
44      csrr s5,3074
45      csrr s4,3073
46      jal F3
47      csrr t0,3073
48      csrr t1,3074
49      sub s4,t0,s4 # s4 = time
50      sub s5,t1,s5
51
52      csrr s7,3074
53      csrr s6,3073
54      jal F4
55      csrr t0,3073
56      csrr t1,3074
57      sub s6,t0,s6 # s6 = time
58      sub s7,t1,s7
59
60      csrr s9,3074
61      csrr s8,3073
62      jal F5
63      csrr t0,3073
64      csrr t1,3074
65      sub s8,t0,s8 # s8 = time
66      sub s9,t1,s9
67
68      csrr s11,3074

```

```

69  csrr s10,3073
70  jal F6
71  csrr t0,3073
72  csrr t1,3074
73  sub s10,t0,s10 # s10 = time
74  sub s11,t1,s11
75
76  li a7, 10
77  ecall
78
79 .data
80 .LC0:
81     .word    0
82     .word    1074790400

```

Listing 3: teste12.s usando a otimização -O1

Com a otimização -O1, foram obtidos os seguintes valores para os registradores, considerando os mesmos registradores que foram apresentados anteriormente.

Registers	Floating Point	Control and Status
Name	Number	Value
zero	0	0
ra	1	4194524
sp	2	2147479548
gp	3	268468224
tp	4	0
t0	5	1180349237
t1	6	56
t2	7	0
s0	8	793
s1	9	6
a0	10	0
a1	11	0
a2	12	0
a3	13	0
a4	14	0
a5	15	268501040
a6	16	0
a7	17	10
s2	18	714
s3	19	6
s4	20	714
s5	21	6
s6	22	722
s7	23	6
s8	24	720
s9	25	6
s10	26	1444
s11	27	10
t3	28	0
t4	29	0
t5	30	0
t6	31	0
pc		4194548

Figura 5: Registradores da otimização -O1

Observe que os testes com -O0 resultaram em um maior tempo de execução. Isso ocorre porque o -O0 não otimiza nada, ele apenas faz uma tradução literal do C para o Assembly. Ao comparar com -O1, o número

de instruções caiu drasticamente, tornando também o tempo de execução menor.

Em conclusão, a equipe observou que programar em alto nível (C) nos afasta do hardware, mas o compilador atua como uma ponte crucial. Sem otimização (-O0), o código é seguro, mas ineficiente. Com otimização (-O1 em diante), o GCC reescreve a lógica para explorar os registradores do RISC-V ao máximo. Além disso, confirmamos que operações com ponto flutuante (f6) são muito mais custosas para o processador em termos de ciclos do que a matemática de inteiros.

Abaixo, observam-se as principais diferenças entre as otimizações -O0, -O1, -O2, -O3 e -Os:

- **-O0 (Nenhuma otimização):** É o padrão. Foca em compilação rápida e em produzir um código que seja fácil de debugar. O mapeamento entre as linhas em C e as instruções em Assembly é direto.
- **-O1 (Otimização básica):** Tenta reduzir o tamanho do código e o tempo de execução sem gastar muito tempo compilando. Ele começa a remover código morto e tenta colocar variáveis em registradores em vez de na pilha.
- **-O2 (Otimização recomendada):** Ativa quase todas as otimizações suportadas que não envolvem um compromisso entre espaço e velocidade. O compilador não faz "desenrolamento" de loops, mas a performance geral melhora drasticamente em relação ao -O1.
- **-O3 (Otimização agressiva / Velocidade):** Otimiza o código pensando exclusivamente em velocidade. Ativa tudo do -O2 e adiciona estratégias pesadas como *function inlining* (copiar o corpo de uma função para dentro de quem a chamou) e *loop unrolling*. É por isso que ele é mais rápido, mas pode gerar um arquivo binário bem maior.
- **-Os (Otimização de tamanho):** Ativa todas as flags do -O2 que normalmente não aumentam o tamanho do código. Além disso, realiza ajustes finos para que o executável ocupe o menor espaço possível em disco e na memória (excelente para sistemas embarcados).

3 Cálculo das raízes da equação de segundo grau

3.1 Procedimento *fx*

O procedimento *fx* é responsável por aplicar um valor real qualquer, armazenado em *fa0*, em uma função quadrática com coeficientes A, B e C, armazenados, respectivamente, em *fa1*, *fa2* e *fa3*. Naturalmente, isso é feito apenas com algumas operações em ponto flutuante, como pode ser visto a seguir:

```
1 # float fx (float x, float a, float b, float c)
2
3 # fa0 ← x
4 # fa1 ← a
5 # fa2 ← b
6 # fa3 ← c
7
8 FX:
9 fmv.s ft0, fa3 # ft0 = c
10 fmul.s ft1, fa2, fa0
11 fadd.s ft0, ft0, ft1 # ft0 = c + xb
12 fmul.s ft1, fa0, fa0
13 fmul.s ft1, ft1, fa1
14 fadd.s fa0, ft0, ft1 # fa0 = c + xb + xxa
15 ret
```

Listing 4: Procedimento *fx*

3.2 Procedimento plot

A função plot é responsável por desenhar o gráfico $f(x)$ na tela gráfica do RARS (o Bitmap Display), em que a origem é disposta sempre no centro. Além disso, deve-se calcular automaticamente a escala e o tamanho dos dois eixos, de modo que, para qualquer função, tenhamos uma representação verossímil da curva. Por último, no código, escolheram-se os registradores fa0, fa1 e fa2 como, respectivamente, os coeficientes A, B e C.

3.2.1 Cálculo da escala e tamanho dos semieixos

Essa seção foi, com certeza, a mais complicada do procedimento. Logicamente, haverá casos em que uma escala será insuficiente, visto que o gráfico pode, por exemplo, estar fora da tela ou não se parecer com uma parábola. Dessa forma, optou-se por um cálculo centrado, sobretudo, no ponto médio da função.

Para funções quadráticas, é vital recordar as fórmulas de ponto médio:

$$X_m = \frac{-b}{2a} \quad (1)$$

Em código, resulta nas seguintes instruções:

```
1 ##### CALCULAR PONTO MEDIO #####
2
3 # Xm = -b/2a
4 # ft0 = Xm
5
6 # o código abaixo simplesmente calcula o Xm com a fórmula acima
7
8     fdiv.s ft0, fa1, fa0 # ft0 = b/a
9     li t0, 2 # t0 = 2
10    fcvt.s.w ft1, t0 # converte o inteiro 2 no float 2.0
11    fdiv.s ft0, ft0, ft1 # ft0 = ft0 / 2
12    li t0, -1 # t0 = -1
13    fcvt.s.w ft1, t0 # converte o inteiro -1 no float -1.0
14    fmul.s ft0, ft0, ft1 # ft0 = -ft0
```

Listing 5: Código de cálculo do X_m

$$Y_m = \frac{-(b^2 - 4ac)}{4a} \quad (2)$$

Da mesma forma, resulta no código:

```
1 # Ym = -(b^2 - 4ac)/4a
2 # ft1 = Ym
3
4 # da mesma forma, calcula o Ym com a fórmula acima
5
6     fmul.s ft1, fa1, fa1 # ft1 = b
7     li t0, 4 # t0 = 4
8     fcvt.s.w ft2, t0 # converte o inteiro 4 para o float 4.0
9     fmul.s ft2, ft2, fa0 # ft2 = 4*a
10    fmul.s ft2, ft2, fa2 # ft2 = 4*a*c
11    li t0, -1
12    fcvt.s.w ft3, t0
13    fsub.s ft1, ft1, ft2 # ft1 = b^2 - 4ac
14    fmul.s ft1, ft1, ft3 # ft1 = -ft1
15    li t0, 4
```

```

16      fcvt.s.w ft2 , t0
17      fmul.s ft2 , ft2 , fa0 # ft2 = 4a
18      fdiv.s ft1 , ft1 , ft2 # ft1 = ft1/4a

```

Listing 6: Código de cálculo do Ym

Com esses resultados, sabe-se onde a curva, geralmente, estará localizada. Para melhor visualização, escolheu-se que cada semieixo será o dobro do ponto médio. Caso contrário, o gráfico estaria no limite da tela. Ademais, vale ressaltar algumas exceções que precisam ser tratadas:

- $X_m = 0$ e $Y_m = 0$: aqui, há o caso em que a curva se localiza exatamente na origem, o esperado de funções em que o único coeficiente não-nulo será o A. Assim, incluiu-se, no .data, uma escala padrão pequena, ideal para qualquer função dessa forma;
- $X_m = 0$ ou $Y_m = 0$: aqui, optou-se por simplesmente copiar o valor da escala não-nula para a nula. Isso é suficiente para esse caso, visto que, mesmo com uma diferença grande entre as escalas, o gráfico ainda será visível.

Por fim, é importante destacar que escolheram-se valores arbitrários para multiplicar a escala. Isso foi feito para garantir que os exemplos ao final do documento tenham maior proporção e beleza.

Para funções lineares, optou-se por um procedimento diferente, já que retas não possuem pontos médios. Com isso, a fórmula geral será:

$$ESCALA = 2 \times B \times C \quad (3)$$

Sabe-se que, em uma função afim, o coeficiente C será onde a reta interceptará o eixo y. Por isso, a base da fórmula será esse dado, pois, depois da raiz real, é a informação mais importante que o gráfico revela. Além disso, para garantir que, independentemente das proporções da função, o desenho se mantenha na tela, multiplica-se o C por B e por 2, e, para configurar a inclinação da reta, modifica-se o X de início e final de acordo com o coeficiente angular. Também é importante considerar funções constantes, que, com o coeficiente B igual a 0, podem zerar a escala caso não haja rotinas específicas. De qualquer maneira, com os devidos cuidados, não é necessário uma lógica totalmente diferente para essa categoria de funções.

Com tudo isso feito, foi possível definir uma escala que, embora imperfeita, é capaz de representar fielmente as funções testadas durante o laboratório. De qualquer maneira, ao longo prazo, há várias formas de aperfeiçoá-la.

3.2.2 Desenho do gráfico

Com a escala definida, deve-se elucidar a lógica de desenho. Em primeiro lugar, define-se que o gráfico terá 280x200 pixels, um pouco menor que a tela, já que uma porção deve ser usada para imprimir as raízes. Isso implica que cada semieixo X terá 140 pixels, enquanto cada semieixo Y, 100 pixels. Dessa maneira, o valor real de cada pixel será obtido a partir das expressões a seguir:

$$ValorPixel_x = \frac{ESCALA_x}{140} \quad (4)$$

$$ValorPixel_y = \frac{ESCALA_y}{100} \quad (5)$$

Dáí, pode-se começar a desenhar o gráfico. Isso será feito por meio de um loop com 280 iterações, de 300 a 20, para cada um dos pixels do eixo X. No entanto, além do pixel, deve-se manter o registro do valor real associado a ele. Para isso, será mantido, em um registrador separado, o maior valor desse eixo, que será, a cada iteração, subtraído pelo valor real unitário. Posteriormente, o conteúdo desse registrador será passado pelo procedimento fx , que retornará o $f(x)$ da função naquele ponto. Por último, esse resultado será convertido para um valor em pixels que respeite as dimensões da tela utilizando a fórmula a seguir, obtida por inúmeros testes e cálculos de proporção:

$$F(x)(Pixels) = \frac{(ESCALA_y - F(x)(Real)) \times 100}{ESCALA_y} + 20 \quad (6)$$

Posteriormente, deve-se lembrar que, principalmente em funções de crescimento rápido, pode haver valores de X pequenos que geram um $f(X)$ grande. Para resolver isso, utilizaram-se diversas condicionais. Para concluir, a cada iteração, pegam-se dois desses pontos e utiliza-se a *ecall* 147, responsável por desenhar linhas, de modo a obter uma curva de alta qualidade. Assim, ao terminar a lógica, obteve-se a seguinte sequência de instruções:

```

1 # ft2 armazena a escala
2 # ft4 eh o valor inicial
3 # ft5 eh o valor que deve ser subtraido de ft4 a cada loop
4
5 # a cada iterecao teremos dois pontos, para que possamos construir a reta
6
7 GRAFICO_LOOP:      blt s0, s1, EXIT
8
9
10      fmv.s fa0, ft4 # em fa0 teremos o f(x) atual
11      jal FX
12
13      mv a0, s0
14
15      addi s0, s0, -1 # atualiza o contador do loop
16      fsub.s ft4, ft4, ft5 # atualiza o x real
17
18 ### CONVERSAO DE VALOR REAL PARA VALOR EM PIXEL ###
19
20 # Vpixel = ((ESCALA - f(x) . 100) / ESCALA) + 20
21
22 # Usamos essa formula para garantir que o ponto estara dentro dos limites da
   memoria de video.
23
24      fsub.s fa0, ft2, fa0
25      li t1, 100
26      fcvt.s.w ft3, t1
27      fmul.s fa0, fa0, ft3
28      fdiv.s fa0, fa0, ft2
29      fcvt.w.s a1, fa0
30      addi a1, a1, 20
31
32 # Calculo do segundo ponto
33
34      fmv.s fa0, ft4
35      jal FX
36      mv a2, s0
37
38      addi s0, s0, -1
39      fsub.s ft4, ft4, ft5
40
41      fsub.s fa0, ft2, fa0
42      li t1, 100
43      fcvt.s.w ft3, t1
44      fmul.s fa0, fa0, ft3
45      fdiv.s fa0, fa0, ft2
46      fcvt.w.s a3, fa0
47      addi a3, a3, 20

```

```

48
49 # As verificacoes a seguir garantem que o ponto estara abaixo do limite do
    eixo Y. Caso nao estejam, simplesmente o ignoramos e passamos para o
    proximo
50
51     li t1, 20
52     blt a1, t1, GRAFICO_LOOP
53     blt a3, t1, GRAFICO_LOOP
54
55     li t1, 220
56     bgt a1, t1, GRAFICO_LOOP
57     bgt a3, t1, GRAFICO_LOOP
58
59     li a4, 0x38
60     li a5, 0
61     li a7, 147
62     ecall # desenhamos a reta
63
64     addi s0, s0, 1 # adicao para que o segundo ponto dessa iteracao
        seja o primeiro da proxima. Isso garante que as linhas sejam
        continuas.
65     fadd.s ft4, ft4, ft5
66     j GRAFICO_LOOP

```

Listing 7: Código para desenhar gráfico de funções quadráticas

Agora, para funções lineares e constantes, optou-se por um processo um tanto diferente. Para construir uma reta, basta que haja dois pontos: fazer 280 iterações seria excessivamente custoso. Por isso, pega-se o $f(x)$ dos limites positivos e negativos do eixo X, aplicando a *ecall* 147 em ambos. Por último, deve-se apenas realizar algumas verificações para assegurar que os valores do eixo Y não excedam o suportado pela VGA. Assim, obteve-se o seguinte código:

```

1  FUNCAO_CONSTANTE:
2
3     li t1, -1
4     fcvt.s.w ft3, t1 # transformando o limite positivo em negativo
5
6     fmul.s fs1, fs3, ft3 # fs3 sera o limite positivo, fs1 o negativo,
        fs2 sera a escala pura
7
8     fmv.s fa0, fs3
9     jal FX
10
11     fsub.s fa0, fs2, fa0 # a logica de conversao de valor real para
        pixel eh a mesma da funcao quadratica
12     li t1, 100
13     fcvt.s.w ft3, t1
14     fmul.s fa0, fa0, ft3
15     fdiv.s fa0, fa0, fs2
16     fcvt.w.s a3, fa0
17     addi a3, a3, 20
18
19     fmv.s fa0, fs1
20     jal FX
21

```

```

22      fsub.s fa0, fs2, fa0
23      li t1, 100
24      fcvt.s.w ft3, t1
25      fmul.s fa0, fa0, ft3
26      fdiv.s fa0, fa0, fs2
27      fcvt.w.s a1, fa0
28      addi a1, a1, 20
29
30      li a4, 0x38
31      li a5, 0
32      li a7, 147
33      ecall # desenhando a reta

```

Listing 8: Código para desenhar gráfico de funções lineares e constantes

Como resultado final, obteve-se um procedimento capaz de representar graficamente funções quadráticas, lineares e constantes das mais diversas magnitudes. Desse modo, o grupo conclui esse procedimento com sucesso.

3.3 Procedimento baskara

O procedimento `int baskara(float a, float b, float c)` encontra as raízes da equação quadrática fornecida $f(x) = ax^2 + bx + c$ através das fórmulas já consolidadas para raízes (r) de polinômios de grau 2 e 1, respectivamente mostradas abaixo:

$$r_1, r_2 = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad (7)$$

$$r = \frac{-c}{b} \quad (8)$$

No tocante ao armazenamento de raízes, são destinadas 4 words na pilha para raízes complexas conjugadas (caso $b^2 < 4ac$) e 2 words para raízes reais distintas ou iguais (caso $b^2 \geq 4ac$); 1 word é separada para a raiz de funções afins (polinômios de grau 1).

A sub-rotina retorna 2 para raízes complexas conjugadas e 1 para raízes reais. Como não foi estabelecido valor de retorno para funções constantes (polinômio de grau 0), optou-se que o procedimento ignorasse tais funções.

```

1 baskara:
2     fcvt.w.s t0, fa0          # t0 = (int)a
3     bne t0, zero, QUADRATICA  # Se a funcao for quadratica
4
5     fcvt.w.s t0, fa1          # t0 = (int)b
6     bne t0, zero, RETAAFIM    # Se a funcao for uma reta
7     ret                      # Ignora funções constantes
8
9 RETAAFIM:
10    # Se a funcao for uma reta
11    fneg.s fa2, fa2           # c = -c
12    fdiv.s ft0, fa2, fa1      # fa0 = -c/b : Raiz da reta
13
14    addi sp, sp, -4           # Prepara a pilha para receber uma raiz
15    fsw ft0, 0(sp)           # Empilha a raiz unica na pilha
16
17    li a0, 1                  # a0= 1(Retorno inteiro da funcao para raizes reais)
18    ret
19

```

```

20 QUADRATICA:
21 # Calculo delta
22 fmul.s fs0 , fa1 , fa1      # fs0 = b^2
23 li t0 , 4
24 fcvt.s.w ft0 , t0          # ft0 = 4
25 fmul.s fs1 , fa0 , ft0      # fs1 = 4*a
26 fmul.s fs1 , fs1 , fa2      # fs1 = 4*a*c
27 fsub.s fs0 , fs0 , fs1      # fs0 = b^2 - 4*a*c = delta
28 fcvt.w.s t0 , fs0           # t0 = (int)fs0 = (int) delta
29 bge t0 , zero , REAIS      # if (delta >= 0) -> Rotina Raizes Reais
30 j COMPLEXAS                # else -> Rotina Raizes Complexas
31
32 REAIS:                      # delta >= 0. Casos Raizes reais
33 fsqrt.s fs0 , fs0           # fs0 = sqrt(fa0) = sqrt(delta)
34 li t0 , 2
35 fcvt.s.w ft0 , t0          # ft0 = 2
36 fmul.s ft0 , ft0 , fa0      # ft0 = 2a
37 fdiv.s fs0 , fs0 , ft0      # fs0 = sqrt(delta) / (2*a)
38 fneg.s fa1 , fa1            # fa1 = -b
39 fdiv.s fa1 , fa1 , ft0      # fa1 = -b / (2*a)
40
41 # Calculo e armazenamento das raizes reais
42 addi sp , sp , -8           # Prepara a pilha para receber 2 words
43 fadd.s ft0 , fa1 , fs0      # ft0 = raiz1 = -b / (2 * a) + sqrt(delta)/(2*a)
44 fsw ft0 , 4(sp)             # empilha raiz1
45 fneg.s fs0 , fs0            # fs0 = -sqrt(delta)/(2*a)
46 fadd.s ft0 , fa1 , fs0      # ft0 = raiz2 = -b / (2 * a) - sqrt(delta)/(2*a)
47 fsw ft0 , 0(sp)             # empilha raiz2
48
49 li a0 , 1                   # a0 = 1 (Retorno inteiro da funcao para raizes reais)
50 ret
51
52 COMPLEXAS:                  # Calculo para delta < 0:
53 fabs.s fs0 , fs0            # fs0 = abs(fs0) = abs(delta)
54 fsqrt.s fs0 , fs0           # fs0 = sqrt(fa0) = sqrt(delta)
55 li t0 , 2
56 fcvt.s.w ft0 , t0          # ft0 = 2
57 fmul.s ft0 , ft0 , fa0      # ft0 = 2a
58 fdiv.s fs0 , fs0 , ft0      # fs0 = sqrt(delta) / (2*a)
59
60 fneg.s fa1 , fa1            # fa1 = -b
61 fdiv.s fa1 , fa1 , ft0      # fa1 = -b / (2*a)
62
63 # Calculo e armazenamento das raizes reais
64 addi sp , sp , -16          # Prepara a pilha para receber 4 words
65 fsw fa1 , 12(sp)            # Empilha parte real da raiz1
66 fsw fs0 , 8(sp)             # Empilha parte imaginaria da raiz1
67 fsw fa1 , 4(sp)             # Empilha parte real da raiz2
68 fneg.s fs0 , fs0            # fs0 = -sqrt(delta)/(2*a)
69 fsw fs0 , 0(sp)             # empilha parte imaginaria da raiz2
70
71 li a0 , 2                   # a0 = 2 (Retorno inteiro da funcao para raizes complexas)
72 ret

```

3.4 Procedimento show

A sub-rotina void `show(int t)` escreve no *Bitmap Display* do *RISC-V Assembler and Runtime Simulator* (RARS) as raízes da função fornecida, por meio das *syscalls* customizadas 102 e 104, usadas, nessa ordem, para "mostrar uma string" e "mostrar um número float IEEE 754 em notação científica".

O parâmetro inteiro `t` define quantas raízes serão mostradas na tela. Para `t=2`, são apresentadas 2 raízes complexas; para `t=1` são mostradas de 1 a 2 raízes, a depender da categoria da função fornecida - 2 raízes para funções de segundo grau e 1 para funções de primeiro grau. Para demais valores de `t` a rotina não mostra nada na tela.

Visto que o procedimento `show` é invocado diretamente após a rotina `baskara` e que é a última sub-rotina da `main`, decidiu-se por realizar a liberação de memória da pilha utilizada no procedimento `baskara`, dentro da própria função `show`.

```

1 show:
2 # Configuracoes para impressao no Bitmap Display
3   li a1, 0      # Define coluna
4   li a2, 223    # Define linha
5   li a3, 0x38   # Define cor de fundo e de frente
6   li a4, 0      # Define frame
7
8   li t0, 1      # Caso baskara diga que ha raizes reais
9   beq a0, t0, showReal # Salte para showReal
10  li t0, 2      # Caso baskara diga que ha raizes complexas
11  beq a0, t0, showImag # Salte para showImag
12  ret          # Tratamento funcao constante
13
14 showReal:
15   fcvt.w.s t0, fa0      # t0 = (int)a
16   bne t0, zero, showRaizDupla # Se a funcao for quadratica
17   # Caso reta afim
18   la a0, r      # Pega string "R = "
19   li a7, 104    # Printa "R = "
20   ecall
21   flw fa0, 0(sp) # Pega raiz na pilha
22   li a7, 102    # Imprime raiz
23   ecall
24   addi sp, sp, 4 # Libera 1 word da pilha
25   ret
26
27 showRaizDupla:
28   la a0, r1      # Pega string "R(1) = "
29   li a7, 104    # Printa "R(1) = "
30   ecall
31
32   flw fa0, 0(sp) # Pega raiz2 na pilha
33   li a7, 102    # Imprime raiz2
34   ecall
35
36   li a1, 0      # Redefine para coluna 0
37   li a2, 231    # Redefine para linha 231
38

```

```

39  la a0, r2    # Pega string "R(2) = "
40  li a7, 104   # Printa "R(2) = "
41  ecall
42
43  flw fa0, 4(sp) # Pega raiz1 na pilha
44  li a7, 102
45  ecall
46
47  addi sp, sp, 8 # Libera 2 words na pilha
48  ret
49
50 showImag:
51  la a0, r1    # Pega string "R(1) = "
52  li a7, 104   # Printa "R(1) = "
53  ecall
54
55  flw fa0, 12(sp) # Pega parte real da raiz1 na pilha
56  li a7, 102    # Imprime parte real da raiz1 na pilha
57  ecall
58
59  flw fa0, 8(sp) # Pega parte imaginaria da raiz1 na pilha
60  li a7, 102    # Imprime parte imaginaria da raiz1 na pilha
61  ecall
62
63  la a0, i     # Pega string "i\n"
64  li a7, 104   # Printa i
65  ecall
66
67  li a1, 0     # Redefine para coluna 0
68  li a2, 231   # Redefine para linha 231
69
70  la a0, r2    # Pega string "R(2) = "
71  li a7, 104   # Printa "R(2) = "
72  ecall
73
74  flw fa0, 4(sp) # Pega parte real da raiz2 na pilha
75  li a7, 102    # Imprime parte real da raiz2 na pilha
76  ecall
77
78  flw fa0, 0(sp) # Pega parte imaginaria da raiz2 na pilha
79  li a7, 102    # Imprime parte imaginaria da raiz2 na pilha
80  ecall
81
82  la a0, i     # Pega string "i\n"
83  li a7, 104   # Printa i
84  ecall
85
86  addi sp, sp, 16 # Libera 4 words na pilha
87  ret

```

Listing 10: Código da sub-rotina *show*

3.5 Programa main

O objetivo da *main* é apenas amalgamar todos os procedimentos feitos anteriormente, além de criar uma interface amigável com o usuário. O código final pode ser visto logo abaixo:

```
1 .data
2 r1: .string "R(1) = "
3 r2: .string "R(2) = "
4 i: .ascii "i"
5 r: .string "R = "
6
7 pergunta_coeficiente_A: .string "Digite o coeficiente A: "
8 pergunta_coeficiente_B: .string "Digite o coeficiente B: "
9 pergunta_coeficiente_C: .string "Digite o coeficiente C: "
10
11 .text
12 MAIN:
13
14     la a0, pergunta_coeficiente_A # carrega o texto do coeficiente A
15     li a7, 4 # ecall para escrita de string
16     ecall
17
18     li a7, 6 # ecall para leitura de float
19     ecall
20     addi sp, sp, -12 # reserva 3 words na pilha para armazenar os coeficientes
21                     # entre os procedimentos
22     fsw fa0, 0(sp) # armazena o A
23
24     la a0, pergunta_coeficiente_B # carrega o texto do coeficiente B
25     li a7, 4
26     ecall
27
28     li a7, 6
29     ecall
30     fsw fa0, 4(sp) # armazena o B
31
32     la a0, pergunta_coeficiente_C
33     li a7, 4
34     ecall
35
36     li a7, 6
37     ecall
38     fsw fa0, 8(sp) # armazena o C
39
40     flw fa0, 0(sp)
41     flw fa1, 4(sp)
42     flw fa2, 8(sp) # carrega os tres coeficientes para plotar o gráfico
43
44     jal ra, plot
45
46     flw fa0, 0(sp)
47     flw fa1, 4(sp)
48     flw fa2, 8(sp) # carrega os três coeficientes para imprimir as raizes
49     reais
```

```

49  jal ra, baskara # Chama int baskara
50  jal ra, show
51
52  addi sp, sp, 12 # restaura o stackpointer ao valor original
53
54  j MAIN # volta o inicio para que o usuario insira outra funcao

```

Listing 11: Código da função *main*

Em primeiro lugar, há algumas adições no *.data*, strings que serão impressas ao longo da execução. Logo após isso, há varias instruções triviais, apenas de leitura e escrita de dados, além de chamadas para os procedimentos descritos anteriormente. Algo interessante a destacar é o uso do registrador *stack pointer*, que se provou um grande aliado no armazenamento de valores e nas interações com a memória. Com os devidos testes feitos, é perceptível que a *main* cumpriu seu papel de integração, de forma que se pode dizer, novamente, que os estudantes foram capazes de finalizar a seção de maneira bem-sucedida.

3.6 Testes finais e cálculos de tempo de execução

O cálculo do tempo de execução segue a fórmula: $t_{exe} = I \times CPI \times T$ em que I é o total de instruções necessárias, CPI é a média de clocks por instrução e T é o período de clock da máquina. Na simulação requisitada, foi considerado um processador RISC-V 32 bits com $f = 1GHz$, que implica um período de $T = 10^{-6}s$; 10 ciclos de clock para instruções de ponto flutuante e 1 ciclo de clock para instruções da ISA de inteiros.

Para funções com raízes reais (exemplos a e b), a função *baskara* se utilizou de 18 instruções de ponto flutuante e 7 instruções da ISA de inteiros, totalizando um tempo de execução de 187 microssegundos, conforme calculado abaixo:

$$t_{exe} = (18 \times 10 + 7 \times 1) \times 10^{-6} = 187 \times 10^{-6}s \quad (9)$$

Para funções com raízes complexas (exemplos c e d), foram usadas 19 instruções de ponto flutuante e 8 instruções da ISA de inteiros, contabilizando um tempo de execução de 198 microssegundos, em conformidade com a conta abaixo:

$$t_{exe} = (19 \times 10 + 8 \times 1) \times 10^{-6} = 198 \times 10^{-6}s \quad (10)$$

Por fim, para retas afins (exemplo e), o algoritmo executou 5 instruções de ponto flutuante e 5 instruções da ISA de inteiros, atingindo um tempo de execução total de 55 microssegundos, assim como estimado abaixo:

$$t_{exe} = (5 \times 10 + 5 \times 1) \times 10^{-6} = 55 \times 10^{-6}s \quad (11)$$

Para finalizar de uma vez por todas a terceira parte do laboratório, os estudantes tiveram de pôr à prova o programa desenvolvido por meio de casos básicos e extremos. A seguir, cada parte aborda, por completo, os exemplos propostos, incluindo suas especificidades e eficiência, especialmente em relação à rotina *baskara*.

3.6.1 a) $F(x) = x^2 - 9.86960440$

Aqui, exemplifica-se uma função com coeficiente B nulo. Assim, espera-se, de imediato, que o gráfico tenha vértice cortando o eixo Y, especificamente na coordenada do coeficiente C. De qualquer maneira, a seguir, pode-se ver a imagem esperada:

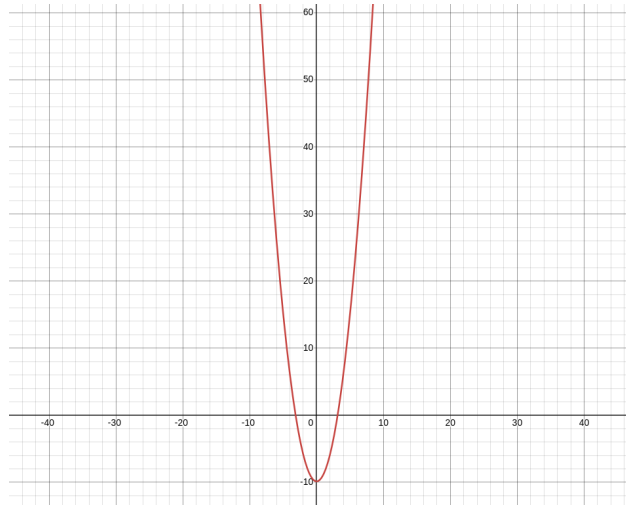


Figura 6: Gráfico esperado da função a)

Aplicando os mesmos coeficientes no algoritmo do grupo, obtém-se este resultado:

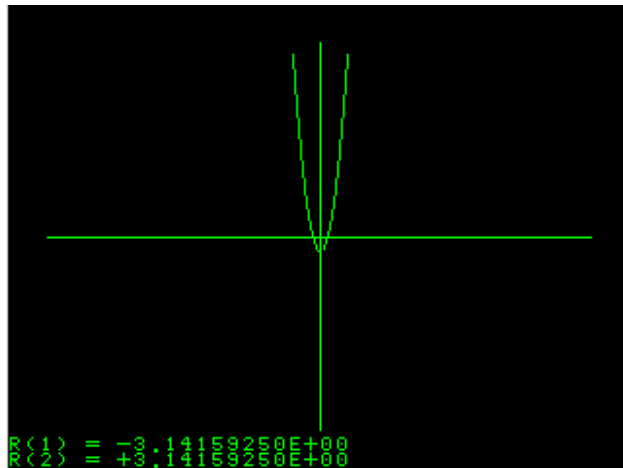


Figura 7: Gráfico obtido da função a)

3.6.2 b) $f(x) = x^2$

Essa é, com certeza, a função mais simples, sendo um exemplo da escala padrão. A seguir, novamente, apresentam-se os gráficos esperados e obtidos:

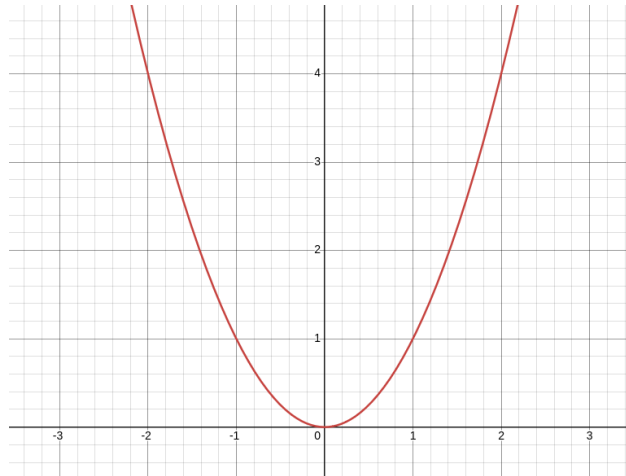


Figura 8: Gráfico esperado da função b)

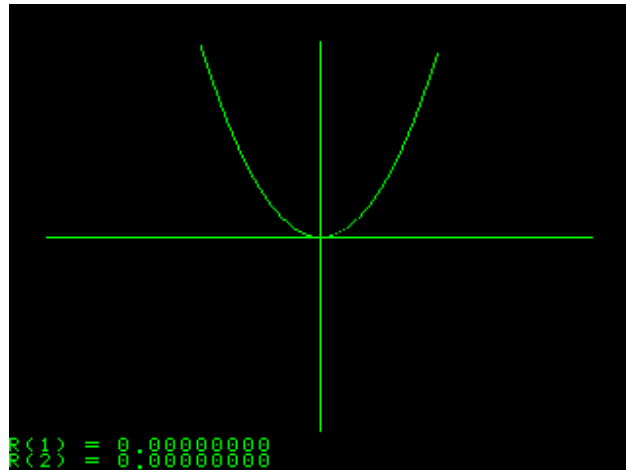


Figura 9: Gráfico obtido da função b)

3.6.3 c) $f(x) = x^2 + 99x + 2459$

Essa é a primeira função quadrática completa. De antemão, é possível ver a imensa proporção de seus coeficientes, que com certeza trouxe problemas ao cálculo de escala. De qualquer forma, a seguir destacam-se os gráficos:

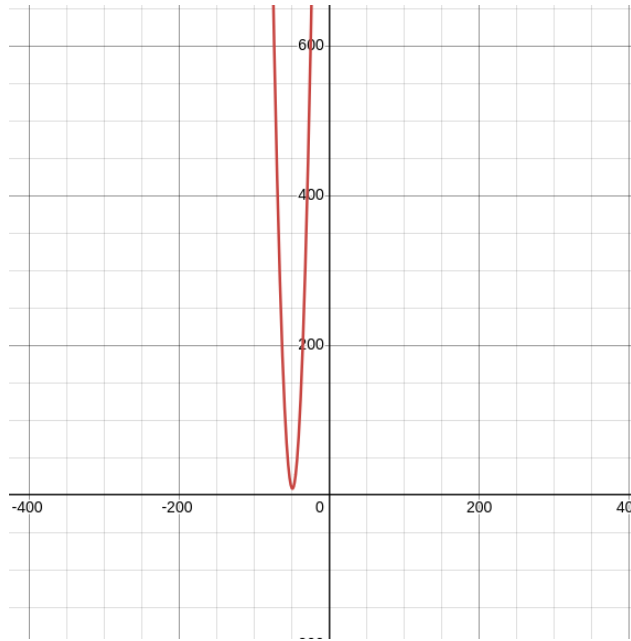


Figura 10: Gráfico esperado da função c)

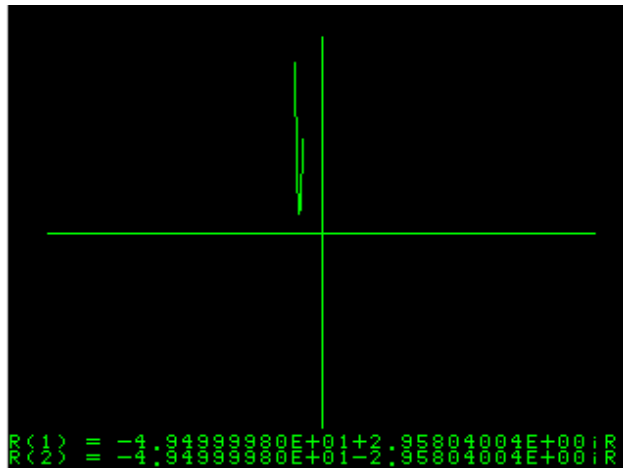


Figura 11: Gráfico obtido da função c)

3.6.4 d) $f(x) = x^2 - 2468x + 33764220$

Esse caso é, obviamente, o mais complicado do experimento, de modo que sequer softwares clássicos do cenário, como o GeoGebra e o Desmos, conseguem representá-lo facilmente. Abaixo, estão as figuras correspondentes:

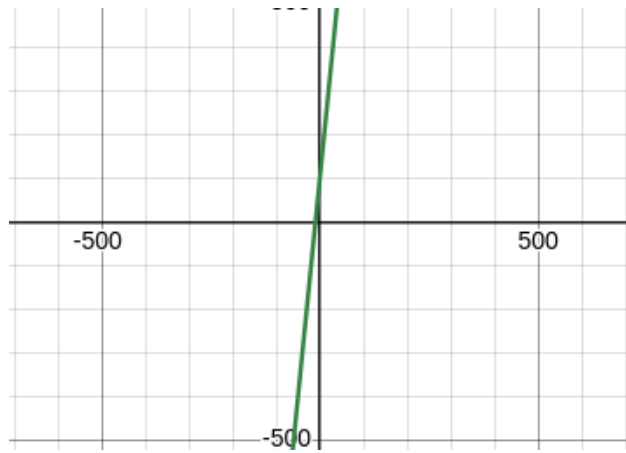


Figura 14: Gráfico esperado da função e)

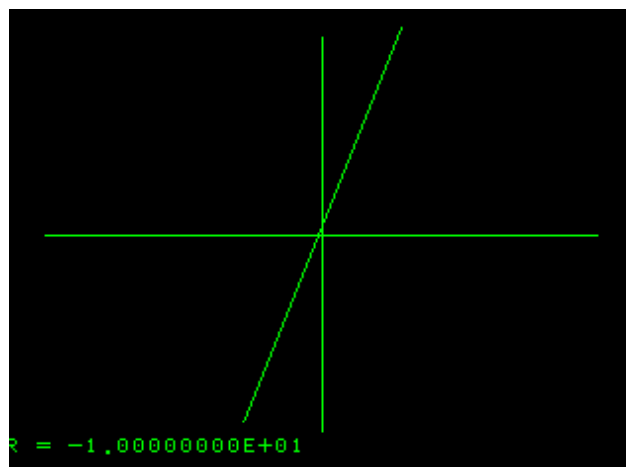


Figura 15: Gráfico obtido da função e)

Por último, a partir deste vídeo, pode-se ver a demonstração completa daquilo descrito nessa seção.